

Lecture 05 NOTES– Thread Synchronization and Context Switching

The Illusion of Concurrency

Concept: Multiple threads *look* like they are running at the same time, but they often take turns on the CPU.

The Conflict: Problems happen when two threads try to change the **same data** at the same time.



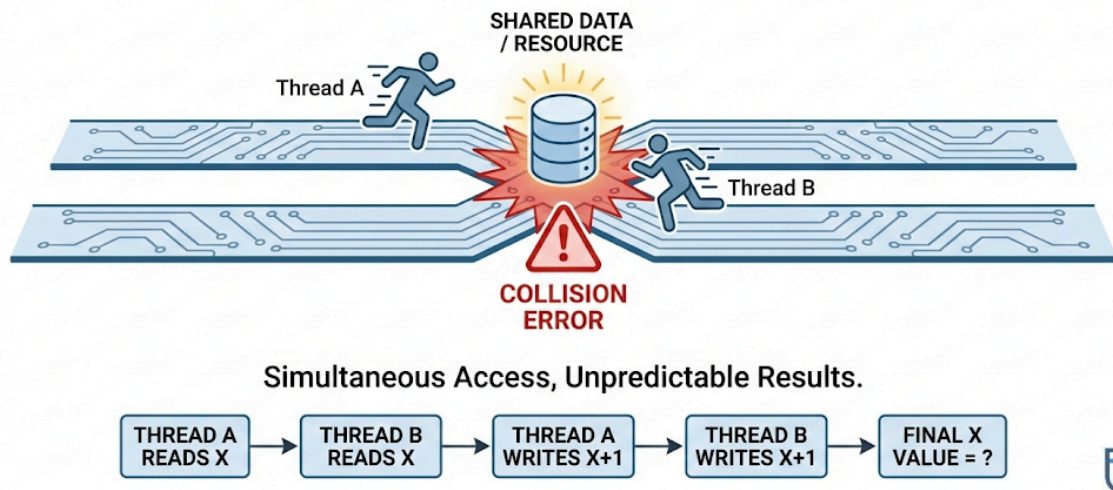
Analogy:

- **Google Docs:** (Synchronized) Two people typing sees each other's changes instantly. Safe.
- **Excel on Shared Drive:** (Race Condition) If two people save at the same time, the last person overwrites the other's work. Unsafe.

Race Conditions

Definition: A bug where the program's output depends on the "luck" of which thread finishes first.

RACE CONDITIONS IN SOFTWARE



Characteristics:

- **Non-deterministic:** The bug doesn't happen every time (hard to repeat).
- **Hard to debug:** Often disappears when you add print statements (Heisenbug).
- **Timing-dependent:** Depends on CPU speed and interruptions.

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++; // ✗ Race condition
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}
```

```
printf("Counter = %d\n", counter); // Wrong value possible
return 0;
}
```

Shared Data Causing Race Conditions

Race conditions only happen in shared memory areas:

Global Variables (Accessible by all).

Heap Memory (Dynamic objects).

Static Variables.

Open Files (Writing to the same file).

Hardware Resources (Printers, ports).

Non-Atomic Operations (Root Cause)

The Issue: Simple code like `counter++` is **not atomic** (not one step).

The Reality: The CPU breaks it into 3 steps:

Read value from memory.

Modify (Add 1).

Write new value back.

The Danger: If a thread is paused (context switch) between step 1 and 3, the data gets corrupted.

Lost Update Problem

Scenario: Two threads try to increment a counter (value 5) at the same time.

Thread A reads 5.

Thread B reads 5 (before A writes back).

Thread A writes 6.

Thread B writes 6 (overwriting A's work).

Result: The value is 6, but it should be 7. Thread A's update was **lost**.

When Synchronization is NOT Needed

You don't need locks/protection for:

Thread-Local Variables: Variables inside a function (Stack memory).

Function Parameters: Passed by value.

Read-Only Data: If no one is writing, everyone can read safely.

Thread-Specific Storage: Variables marked with `__thread` (each thread gets its own copy).

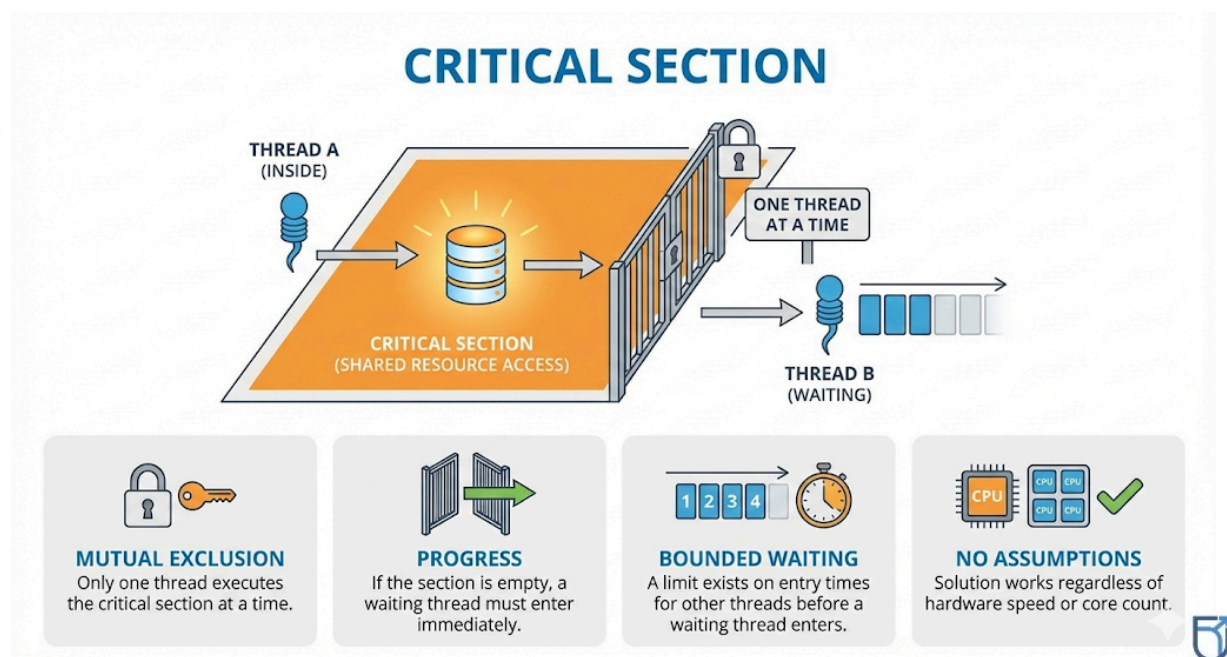
Critical Section

Definition The specific part of the code where a thread accesses a **shared resource** (like a global variable). Only **one** thread can execute this code at a time to prevent errors.

Requirements for a Valid Solution

Mutual Exclusion: If one thread is inside the critical section, no others can enter.

Progress: If the critical section is empty, threads wanting to enter must be allowed in immediately (no deadlock).



Bounded Waiting: A thread cannot wait forever; there must be a limit on how many times others can cut in line (no starvation).

No Assumptions: The solution must work regardless of CPU speed or core count.

Implementation:

```

#include <stdio.h>
#include <pthread.h>

int counter = 0;           // Shared resource
pthread_mutex_t lock;      // Mutex for mutual exclusion

void* increment(void* arg) {

    pthread_mutex_lock(&lock);  // ENTRY SECTION (lock acquired)

    counter++;                  // CRITICAL SECTION
                                // Only one thread executes this

    pthread_mutex_unlock(&lock); // EXIT SECTION (lock released)

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter = %d\n", counter);

    pthread_mutex_destroy(&lock);
    return 0;
}

```

Output

Counter = 2

Mutex (Mutual Exclusion Lock)

Concept

- **Binary Lock:** A lock that has only two states: **Locked** or **Unlocked**.
- **Analogy:** Like a restroom key. Only one person can have it; others must wait.

Behavior

- **Locking (`mutex_lock`):**
 - If **Unlocked**: You take the lock and enter.
 - If **Locked**: Your thread **blocks** (goes to sleep) and waits in a queue. It does not burn CPU power.
- **Unlocking (`mutex_unlock`):**
 - You release the lock.
 - If threads are waiting, the OS **wakes one up** to enter next.

Rule: Only the thread that *locked* the mutex can *unlock* it.

Implementation :

```
#include <stdio.h>
#include <pthread.h>

int shared = 0;           // Shared resource
pthread_mutex_t mutex;    // Mutex lock

void* task(void* arg) {

    pthread_mutex_lock(&mutex); // Lock acquired (only one thread
allowed)

    shared++;                  // CRITICAL SECTION

    pthread_mutex_unlock(&mutex); // Lock released

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex, NULL);
```



```

pthread_create(&t1, NULL, task, NULL);
pthread_create(&t2, NULL, task, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

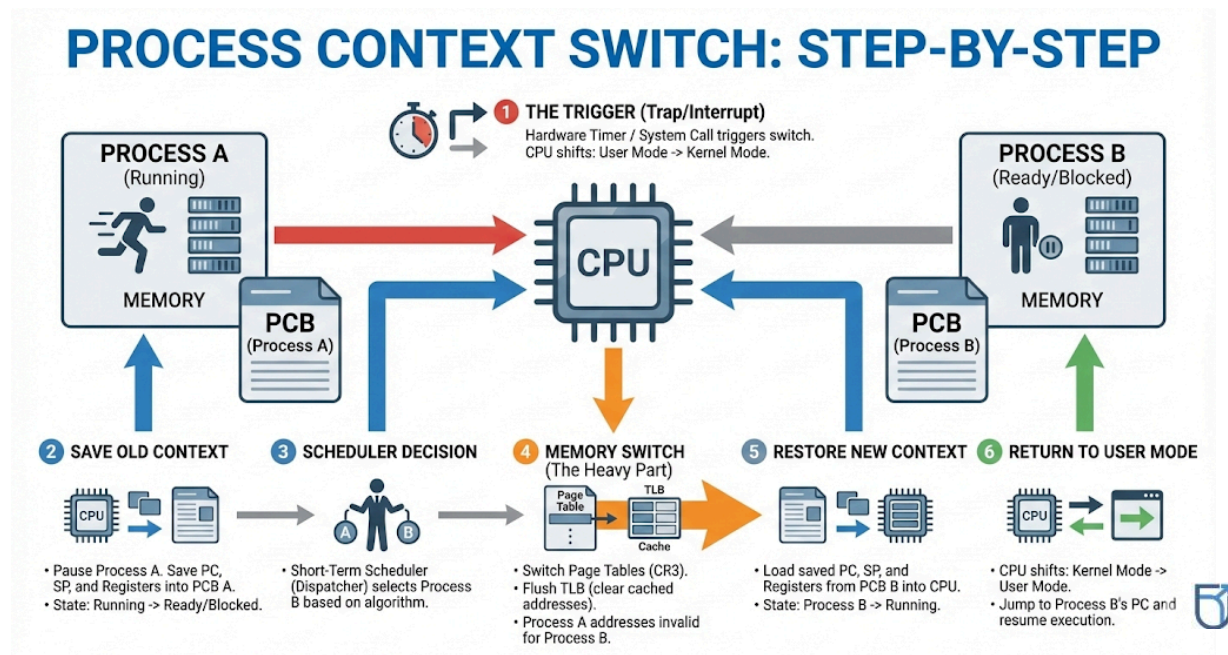
printf("Shared value = %d\n", shared);

pthread_mutex_destroy(&mutex);
return 0;
}

```

Process Context Switch — Step-by-Step

Definition A Context Switch is the mechanism the Operating System uses to stop the currently running process and start executing another one. It is essential for multitasking.



Interrupt

- Save Process A Context
- Scheduler Decision
- Switch Memory Context
- Restore Process B Context
- Resume Execution

Pseudo-Code:

```
void context_switch() {  
  
    save_registers(current_process);    // Save CPU state  
  
    current_process->state = READY;  
    enqueue_ready_queue(current_process);  
  
    next_process = scheduler_select(); // Scheduler decision  
  
    switch_page_table(next_process);    // Address space switch  
  
    restore_registers(next_process);    // Restore CPU state  
  
    run(next_process);  
}
```

The Detailed Procedure:

1. **The Trigger (Trap/Interrupt):**
 - A hardware timer interrupt goes off (time slice over) or the process makes a system call (I/O request).
 - The CPU switches from **User Mode** to **Kernel Mode**.
2. **Save Old Context:**
 - The OS pauses the running process (Process A).
 - It saves the **Program Counter (PC)** (where it stopped), the **Stack Pointer (SP)**, and all general **CPU Registers** into Process A's **PCB** (Process Control Block).
 - Process A's state is updated from "Running" to "Ready" or "Blocked."
3. **Scheduler Decision:**
 - The **Short-Term Scheduler** (Dispatcher) runs and selects the next process to run (Process B) based on the scheduling algorithm.
4. **Memory Switch (The Heavy Part):**
 - **Switch Page Tables:** The OS points the CPU's memory management unit (MMU) to Process B's page directory (changing the **CR3** register on x86).
 - **Flush TLB:** The **Translation Lookaside Buffer (TLB)**, which caches memory addresses, must be cleared (flushed). This is because Process A's addresses are meaningless to Process B.
5. **Restore New Context:**

- The OS loads the saved register values, PC, and SP from Process B's **PCB** back into the hardware CPU registers.
- Process B's state updates to "Running."

6. Return to User Mode:

- The CPU switches back to **User Mode** and jumps to the address in the Program Counter to resume Process B exactly where it left off.

Implementation:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;           // ✓ Critical Section
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

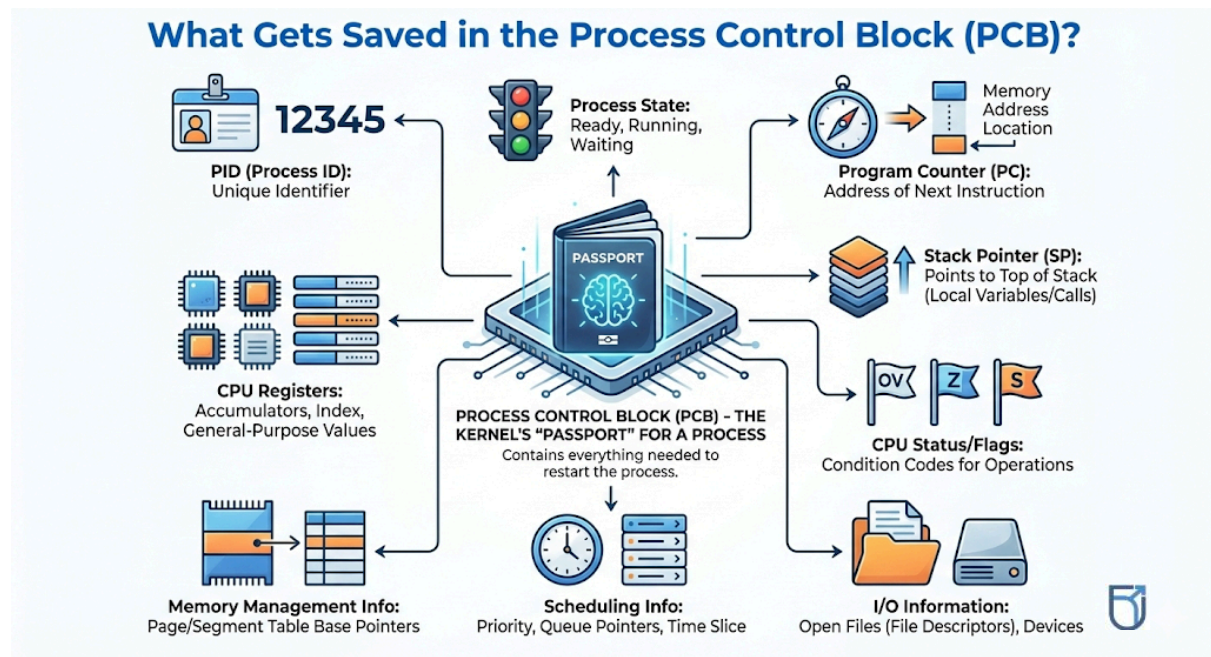
    printf("Counter = %d\n", counter); // Correct
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Output:

```
Counter = 200000
```

What Gets Saved in the PCB?

The **Process Control Block (PCB)** is a data structure in the kernel that acts as the "brain" or "passport" for a process. It contains everything needed to restart the process.



- **PID (Process ID):** The unique number identifying the process.
- **Process State:** Current status (Ready, Running, Waiting).
- **Program Counter (PC):** The address of the *next* instruction to be executed.
- **CPU Registers:** Values of accumulators, index registers, and general-purpose registers.
- **Stack Pointer (SP):** Points to the top of the process's stack (holds local variables/function calls).
- **CPU Status/Flags:** Condition codes (overflow, zero flag, etc.) needed for math operations.
- **Memory Management Info:** The "Base" pointer for Page Tables or Segment Tables.
- **Scheduling Info:** Priority level, queue pointers, time slice limits.
- **I/O Information:** List of open files (file descriptors) and allocated devices.

Hidden Costs of Process Switching

A context switch isn't just about saving registers; the **indirect costs** are often much higher than the direct work.

1. **Register Save/Restore (Direct Cost):**
 - Moving data between CPU registers and RAM takes time, but this is relatively fast (microseconds).
2. **TLB Flush (The Memory penalty):**
 - The TLB is a super-fast cache for memory addresses. When you switch processes, the old addresses are invalid.
 - The OS must **flush** (delete) the TLB entries.

- **Result:** The new process starts with an empty TLB. The first few memory accesses will be very slow as the CPU has to walk the page tables again to refill the TLB.
- 3. **Cache Pollution / Cold Cache Effect (The Performance Killer):**
 - **Hot Cache:** Process A had all its relevant data in the L1/L2/L3 CPU Cache (super fast).
 - **Context Switch:** Process B starts running and pulls *its* data into the cache, overwriting Process A's data.
 - **Cold Cache:** When you switch *back* to Process A later, its data is gone. It faces a "Cold Cache" and must fetch everything from slow RAM again.

Why Process Switch is Expensive

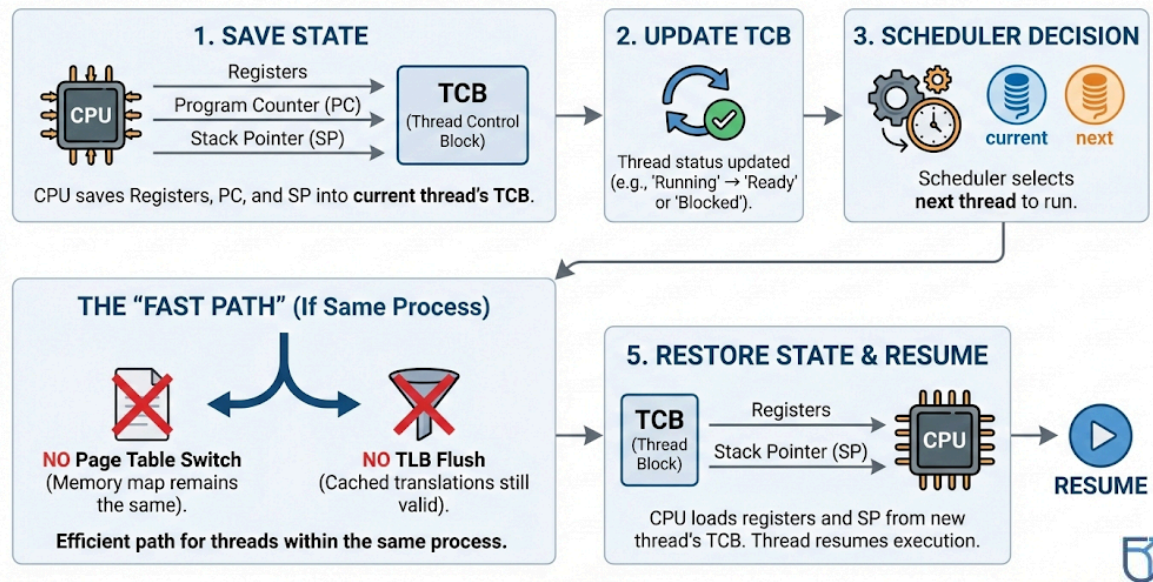
Comparing Process Switching to Thread Switching explains the cost:

1. **Different Address Spaces:**
 - Processes are isolated. Switching requires changing the entire **Virtual Memory map** (Page Tables).
 - *Note:* Thread switching is cheaper because threads share the same memory; they don't need to switch page tables.
2. **Mandatory TLB Flush:**
 - Because the memory map changes, the TLB *must* be wiped. This forces the CPU to do expensive memory translations immediately after the switch.
3. **Large PCB:**
 - A Process Control Block is much larger and contains more administrative data than a Thread Control Block (TCB), taking more time to save and load.

Thread Context Switch — Steps

Key Difference: Unlike processes, threads within the same application **share the same address space** (memory). This sharing is the secret to their speed.

THREAD SWITCHING PROCEDURE: A FAST PATH DIAGRAM



The Procedure:

1. Save State:

- The CPU saves the **Registers**, **Program Counter (PC)**, and **Stack Pointer (SP)** of the current thread into its **Thread Control Block (TCB)**.

2. Update TCB:

- The thread's status is updated (e.g., from "Running" to "Ready" or "Blocked").

3. Scheduler Decision:

- The scheduler selects the next thread to run.

4. The "Fast Path" (If Same Process):

- No Page Table Switch:** The memory map remains exactly the same.
- No TLB Flush:** The cached memory translations are still valid, so the TLB is not wiped.

5. Restore State:

- The CPU loads the registers and stack pointer from the new thread's TCB.

6. Resume:

- Execution continues immediately.

Pseudo-Code

```
void thread_context_switch() {  
  
    save_registers(current_thread);    // Save thread state  
  
    current_thread->state = READY;  
    enqueue_ready(current_thread);  
}
```

```

    next_thread = scheduler_select();

    restore_registers(next_thread);    // Restore new thread state

    run(next_thread);
}

```

Process vs. Thread Switch

A comparison of the overhead involved in switching.

Feature	Process Switch	Thread Switch
Registers	Save/Restore	Save/Restore
Page Table	Switch (Expensive)	No Switch (Instant)
TLB	Flush (Performance Hit)	No Flush (Maintains Speed)
Cache	Cold (New data needed)	Mostly Warm (Shared data)
Time Estimate	~3–5 μ s (Microseconds)	~0.3–0.5 μ s (Microseconds)

Why Thread Switch is Faster

1. **No TLB Flush:** Since the memory address map doesn't change, the CPU doesn't have to relearn where data is stored.
2. **No Page Table Change:** The Memory Management Unit (MMU) settings stay untouched.
3. **Shared Cache:** Because threads share global variables and heap memory, the data currently in the high-speed CPU cache is often useful for the next thread ("Warm Cache").
4. **Smaller Control Block:** Saving a TCB is faster than saving a massive PCB (which includes file handles, permissions, etc.).
5. **Same Address Space:** No complex virtual memory context switching is required.

Overhead Analysis (CPU Cycles)

To quantify the difference in terms of CPU work units (cycles):

- **Process Switch:** ~2000–5000 cycles.
 - *High Cost:* Due to memory management overhead and cache misses.
- **Thread Switch:** ~850–1500 cycles.
 - *Low Cost:* Mostly just saving/loading registers.

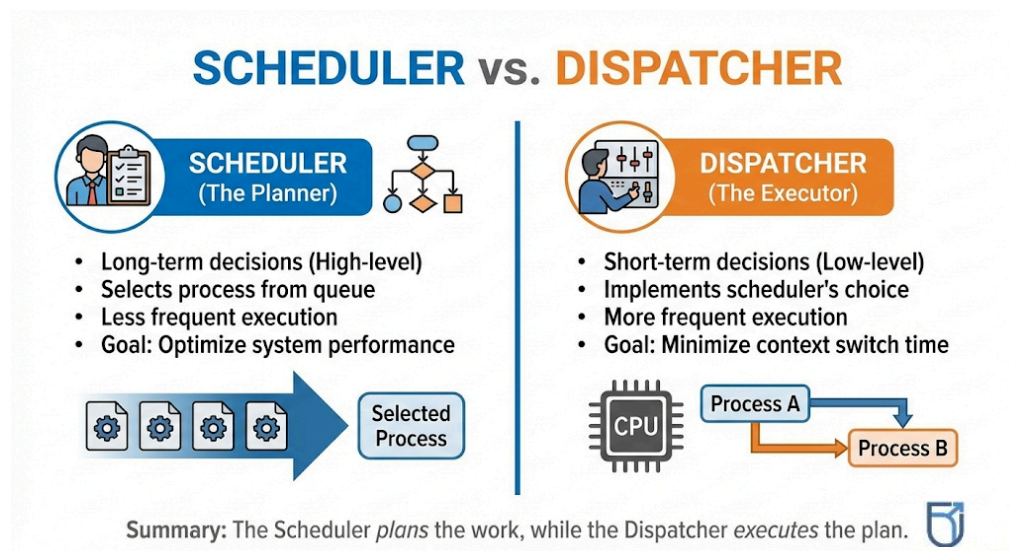
Conclusion: Thread switching is roughly **2–5 times faster** than process switching.

Scheduler vs. Dispatcher

These two components work together to manage CPU execution, but they have distinct roles.

1. The Scheduler (The "Boss")

- **Role:** The decision maker.
- **Action:** It selects *which* process or thread goes next.
- **Mechanism:** It is pure software logic (algorithms like Round Robin or Priority). It looks at the "Ready Queue" and picks a PCB or TCB.
- **Output:** It passes its decision to the Dispatcher.



2. The Dispatcher (The "Worker")

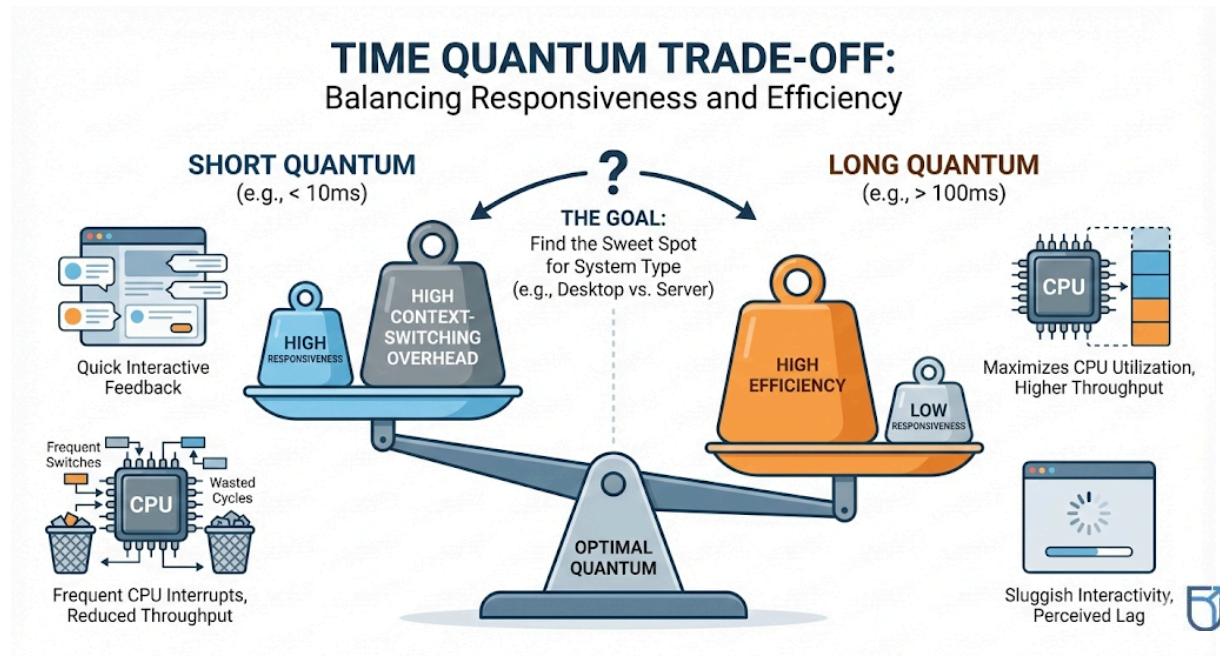
- **Role:** The executor.
- **Action:** It performs the actual "heavy lifting" of the switch.
- **Mechanism:**
 1. **Saves** the old process's registers/state.
 2. **Loads** the new process's registers/state.
 3. **Jumps** to the new location in memory to start execution.
- **Latency:** The time it takes for the dispatcher to stop one process and start another is called **Dispatch Latency**.

Time Quantum Trade-off

In Round Robin scheduling, the "Time Quantum" (or Time Slice) is the fixed amount of time a process is allowed to run before being switched out.

1. Too Small (e.g., 1 ms)

- **Pro:** Extremely high responsiveness (users feel like everything is running instantly).
- **Con: High Overhead.** The CPU spends more time switching (context switching) than actually doing work. Throughput drops significantly.



2. Too Large (e.g., 500 ms)

- **Pro: High Throughput.** The CPU spends almost all its time processing work.
- **Con:** Poor responsiveness. The system feels "laggy" or frozen if a heavy task is running. It essentially behaves like FCFS (First-Come, First-Served).

3. Modern OS Solution

- **Dynamic Quantum:** Most systems (Linux, Windows) use a balanced quantum, typically around **10–20 ms** (or variable based on priority), to get the best of both worlds.

When Context Switch Overhead Matters

In these systems, every microsecond counts, and too much switching can cause failure or money loss.

- **Real-time Systems:** (e.g., Robot control, Airbags) Delays can be dangerous.
- **High-Frequency Trading:** (Stock markets) A nanosecond delay means losing a trade.
- **Game Engines:** Stuttering (lag) ruins the player experience.
- **Databases:** High-frequency locking and switching slows down queries.

- **High-Load Servers:** (e.g., Nginx) Handling millions of requests requires minimizing thread overhead (often solved using Event Loops instead of threads).

When Overhead is Negligible

In these scenarios, the cost of switching is so small compared to the work being done that it doesn't matter.

- **Interactive Apps:** (Word, Browser) A 5 microsecond delay is invisible to a human typing.
- **Batch Jobs:** (Video rendering, Scientific analysis) These run for hours; a few switches don't impact the total time.
- **I/O-Bound Programs:** (File copying) The program spends most of its time waiting for the disk anyway; the CPU switch time is tiny by comparison.
- **Large Time Quantum Systems:** If tasks run for long periods, the switch happens rarely.